

**PROJECT**AI-ENHANCED DEPARTMENTAL STORE
MANAGEMENT SYSTEM**NAME**

- ZAINAB SARFRAZ
- ADEEN TARIQ
- EMAN QASIM

REG NO

- 24-NTU-CS-FL-1289
- 24-NTU-CS-FL-1242
- 24-NTU-CS-FL-1255

SEMESTER

3rd

COURSE CODE/TITLECOC-2074L/DATA STRUCTURES AND
ALGORITHMS-LAB**SECTION/DEPARTMENT**

BSAI/CS

SUBMITTED TO

MAM. KANWAL KHURSHID

SUBMISSION DATE29th Jan, 2025

DEPARTMENTAL STORE MANAGEMENT SYSTEM

ABSTRACT:

The **AI-Enhanced Departmental Store Management System** is a modern software solution designed to automate and streamline the daily operations of departmental stores. Traditional store management relies heavily on manual processes such as recording sales, updating stock, managing prices, and generating bills. These manual methods are not only time-consuming but also prone to errors, leading to financial losses, inventory mismanagement, and customer dissatisfaction. To overcome these challenges, this project integrates programming concepts, data structures, algorithms, and basic artificial intelligence to make store management easier, faster, and more accurate.

In this system, all products are stored with detailed information, including a unique product ID, name, price, available stock, and sales history. This structured organization allows the system to efficiently manage product data and perform operations without errors. The system provides role-based access for three types of users: **Admin, Manager, and Customer**. Each user role is designed to perform specific tasks according to their responsibilities. The Admin can view the complete inventory, update prices, and restock products to maintain adequate stock levels. The Manager can monitor inventory, check low-stock alerts, analyze sales data, and predict future sales trends based on historical records. Customers can browse products, add them to a virtual cart, receive product recommendations, and generate bills with automatic tax calculations. This role-based design ensures that every user can interact with the system according to their needs.

The system uses **vectors, maps, and structures** to efficiently store and manage data. Vectors provide dynamic storage for product lists, maps store product recommendations for enhancing customer experience, and structures organize detailed product information. Algorithms are applied for searching products, predicting future sales, and suggesting related items. The sales prediction module, for example, calculates expected demand for products based on past sales trends, allowing the store to prepare stock in advance. The recommendation system suggests complementary products to customers, helping increase sales and customer satisfaction.

INTRODUCTION:

In today's fast-paced world, departmental stores play an important role in providing a wide range of products to customers in one place. Managing these stores efficiently is a challenging task due to the large volume of products, constant sales, and the need to maintain accurate stock records.

The **AI-Enhanced Departmental Store Management System** is developed to address these challenges by using modern programming concepts such as **data structures, algorithms, and basic artificial intelligence techniques**. This system is designed to automate daily store operations, reduce errors, and provide helpful insights for decision-making.

In this system, every product is stored with essential details such as **product ID, name, price, available stock, and sales history**. The Admin has complete control over the inventory and can perform operations like viewing the inventory, restocking products, and updating product prices. This ensures that the store always has adequate stock and correct pricing. The Manager can monitor inventory levels, receive low-stock alerts, and predict future sales based on historical data. The Customer can browse products, add them to a virtual shopping cart, get recommendations for related products, and generate a detailed bill including taxes. This makes shopping more convenient, personalized, and accurate.

The system uses **vectors, maps, and structures** to organize and store product data efficiently. Vectors allow dynamic storage of product lists, maps store recommendations between related products, and structures organize detailed product information. Algorithms are applied to search for products, forecast sales, and generate recommendations. For example, the sales prediction algorithm calculates future demand based on past sales, helping the store prepare stock accordingly.

The use of **AI concepts** in this system, even at a basic level, improves operational efficiency. Sales prediction helps prevent stock shortages, while recommendations assist customers in finding products they might need. The system also generates accurate bills automatically, reducing errors in pricing and saving time during checkout. By integrating these features, the AI-enhanced system not only supports store staff but also improves the overall shopping experience for customers.

PROBLEM STATEMENT:

Managing a departmental store efficiently is a complex and time-consuming task due to the variety of products, constant sales, and the need to maintain accurate records. Traditional departmental stores often rely on manual methods such as pen-and-paper logs, spreadsheets, or basic billing systems to track inventory, sales, and customer transactions. These methods are not only slow but also prone to errors.

Another major challenge in traditional store management is **inventory control**. Departmental stores carry a wide range of products, each with its own stock levels, prices, and sales trends. Monitoring inventory manually makes it difficult to predict which products will be in demand soon. As a result, stores often face problems like running out of popular items or holding excess stock of low-demand products. This not only affects sales but also increases storage costs. Additionally, manually analyzing sales trends and forecasting demand is extremely difficult without advanced tools, leading to inefficient decision-making and delayed responses to customer needs.

Billing and pricing are also critical aspects of store management that pose challenges in manual systems. Customers expect fast and accurate billing at the checkout, but manual processes are prone to mistakes in price calculations, tax computations, or discounts. Incorrect billing not only affects store revenue but also damages customer trust. Moreover, providing personalized recommendations to customers, which can increase sales and improve customer satisfaction, is nearly impossible without automation and AI support.

Customer experience is another area affected by traditional store management methods. Modern customers expect convenience, speed, and personalized service when shopping. Without an automated system, it is difficult for stores to track customer preferences, suggest related products, or offer a smooth and efficient checkout process. The absence of such features can reduce customer satisfaction and loyalty, making it harder for stores to compete with modern retail outlets and online shopping platforms.

From the management perspective, decision-making is often reactive rather than proactive in traditional stores. Managers may struggle to identify which products are performing well, which items are low in stock, or which products may be in high demand in the future. This lack of predictive insight makes it difficult to plan restocking, pricing strategies, or promotional activities effectively.

OBJECTIVES:

The objectives of this system are:

✦ **Automate inventory management:**

- Keep accurate records of all products, including ID, name, price, stock, and sales history.
- Prevent stock shortages and overstocking.

✦ **Provide role-based access for different users:**

- **Admin:** Manage inventory, update prices, restock products.
- **Manager:** Analyze sales, monitor low-stock alerts, predict future demand.
- **Customer:** Browse products, add items to cart, receive recommendations, generate bills.

✦ **Provide AI-driven product recommendations:**

- Suggest related or complementary products to customers.
- Enhance customer experience and increase sales.

✦ **Automate billing and tax calculations:**

- Generate bills automatically with accurate totals and taxes.
- Reduce manual errors and save time during checkout.
- Ensure transparency in transactions and improve customer satisfaction.

✦ **Improve overall store efficiency and customer satisfaction:**

- Streamline store operations.
- Ensure accurate records, smooth billing, and personalized shopping experience.
- Support data-driven decisions and proactive store management.

DATA STRUCTURES USED:

In the development of the **AI-Enhanced Departmental Store Management System**, the selection of appropriate data structures plays a vital role in ensuring efficiency, accuracy, and smooth system performance. This system manages a large amount of data such as product information, stock levels, sales records, recommendations, and customer purchases. To handle this data effectively, **Structures, Vectors, and Maps** are used in combination. Each data structure serves a specific purpose and together they form a strong backbone for the system.

Structures:

In this system, a **Product structure** is defined to store all the information related to a product. This includes the product ID, name, price, available stock, and sales history.

- ✦ **Purpose:** The structure is used to group all related information of a product into a single unit. Instead of handling separate variables for product ID, name, price, and stock, the structure allows the system to manage all product-related data together.
- ✦ **Product storage:** Each Product structure stores:
 - **Product ID:** Uniquely identifies each product.
 - **Product Name:** Used for searching and billing.
 - **Price:** Used for calculating total bill.
 - **Stock:** Keeps track of available quantity.
 - **Sales History:** Stores previous sales data, which is later used for AI-based sales prediction.
- ✦ **Advantages:**
 - Organizes related information efficiently.
 - Makes code readable and maintainable.
 - Facilitates easy access and modification of product data.
 - Supports AI features like sales prediction using stored sales history.
 - Simplifies passing product data to functions.

Vectors:

The **inventory** in this system is implemented as a vector of Product structures.

- ✦ **Purpose:** The vector stores all products available in the store dynamically. Since the number of products can change over time, vectors are preferred over arrays because they do not require a fixed size.
- ✦ **Operations:** The inventory vector is used in multiple system operations:
 - Displaying product list
 - Searching products.
 - Sales forecasting and low stock alerts.
 - Updating stock and prices.
- ✦ **Advantages:**
 - Supports fast access to elements using indices.
 - Can grow or shrink dynamically based on the number of products.
 - Provides functions like push_back, size, and erase, which simplify inventory management.

Maps:

In this system, maps are used to implement **product recommendations** and **shopping carts**.

- ✦ **Purpose:**
 - **Recommendations:** Each product name is used as a **key**, and its recommended related product is stored as the **value**. This simulates an AI-based recommendation system where customers receive suggestions while shopping. When a customer adds Milk to the cart, the system recommends Bread automatically.
 - **Shopping Cart:** The cart stores products selected by the customer along with their quantities. **Key** (Product Name), **Value** (Quantity selected)
- ✦ **Advantages:**
 - Allows fast searching, insertion, and retrieval of data using keys.

- Ensures unique keys, which prevents duplicate entries.
- Simplifies implementation of recommendation and billing systems.
- Improves customer experience using smart suggestions.

Combined Usage in the System:

The integration of structures, vectors, and maps makes the system both efficient and flexible. Structures organize product data, vectors maintain the dynamic inventory, and maps provide quick access to recommendations and cart details. Together, these data structures support multiple functionalities:

- ✦ Efficient inventory tracking and updates.
- ✦ Fast product searching using names.
- ✦ Accurate billing with automatic price and tax calculation.
- ✦ AI-based sales prediction using sales history.
- ✦ Low stock alerts for better decision-making.
- ✦ Smooth interaction between Admin, Manager, and Customer modules.

DIAGRAM:

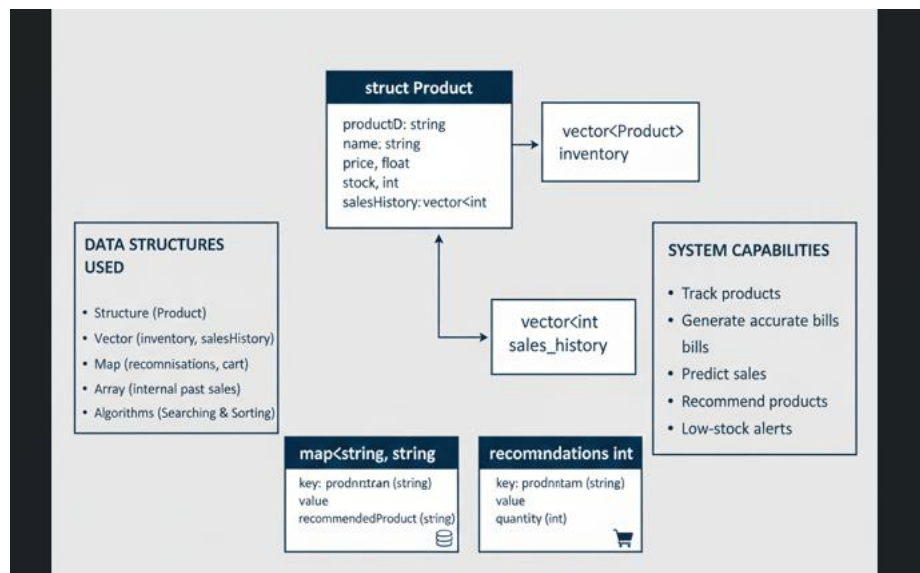


Figure 1 Data Structure Diagram for Products and Inventory

ALGORITHMS USED:

The **AI-Enhanced Departmental Store Management System** uses several simple yet effective algorithms to perform different operations smoothly and efficiently. Instead of using complex algorithms, the system focuses on basic and understandable logic, which makes it reliable, fast, and easy to maintain.

These algorithms help the system in:

- ✦ Searching products quickly
- ✦ Predicting future sales
- ✦ Generating accurate bills
- ✦ Recommending related products
- ✦ Detecting low stock situations

As shown in **Figure 2**

DIAGRAM:

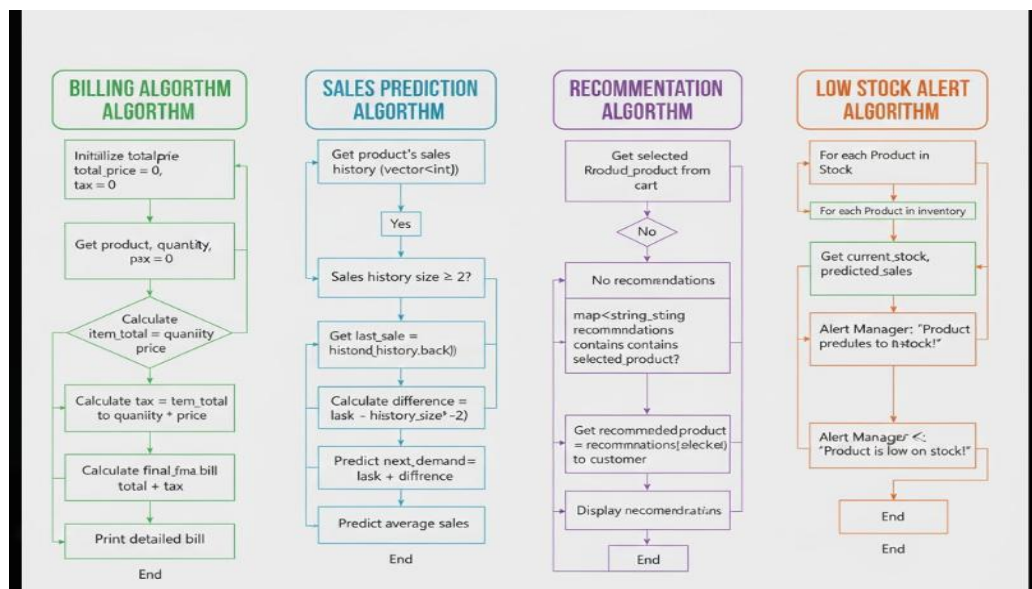


Figure 2 Billing Process Flowchart

Linear Search Algorithm:

In the system, when a user (Admin, Manager, or Customer) enters a product name, the system checks each product one by one in the inventory vector until the required product is found.

✦ Use in the system:

- Product names are stored in a vector.

✦ Role in the System:

- Searching products while adding to cart
- Updating price and stock
- Generating bills

Sorting Algorithm:**✦ Application of sorting:**

- Arrange products by price & sort by names
- View products based on stock level

Sales Prediction Algorithm:

The algorithm calculates the trend using recent sales and estimates future demand. This prediction helps the manager in decision making.

✦ Purpose of Sales Prediction:

- Estimate future product demand
- Support inventory planning
- Avoid overstocking or shortage

✦ Advantages:

- Uses real sales data
- Improves store efficiency

Recommendation Algorithm:

The Recommendation algorithm suggests related products to customers during shopping.

- ✦ Key = Selected product
- ✦ Value = Recommended product

✦ Use of Recommendation Algorithm:

- Enhances customer experience
- Encourages additional purchases

Billing Algorithm:

The Billing algorithm calculates the total bill of the customer.

- ✦ Traverses the shopping cart
- ✦ Calculates cost of each product
- ✦ Adds tax (5%)
- ✦ Displays the final bill

This process is shown in **Figure 2**

✦ Outcome:

- Correct grand total
- Clear bill display
- Smooth checkout process

Low Stock Alert Algorithm:

The Low Stock Alert algorithm helps managers identify products that may run out of stock soon.

✦ Comparison:

- Current stock
- Predicted future sales

✦ Purpose:

- Prevent stock shortage
- Improve inventory control
- Support timely restocking

✦ Benefits:

- Reduces loss of sales
- Helps proactive decision making
- Improves store reliability

SYSTEM DESIGN:

The system design explains how different components of the system interact with each other to manage a departmental store efficiently. The system is divided into multiple modules based on user roles, data storage, and functionalities. It uses data structures and algorithms to make operations fast & organized. As shown in Figure 3

DIAGRAM:

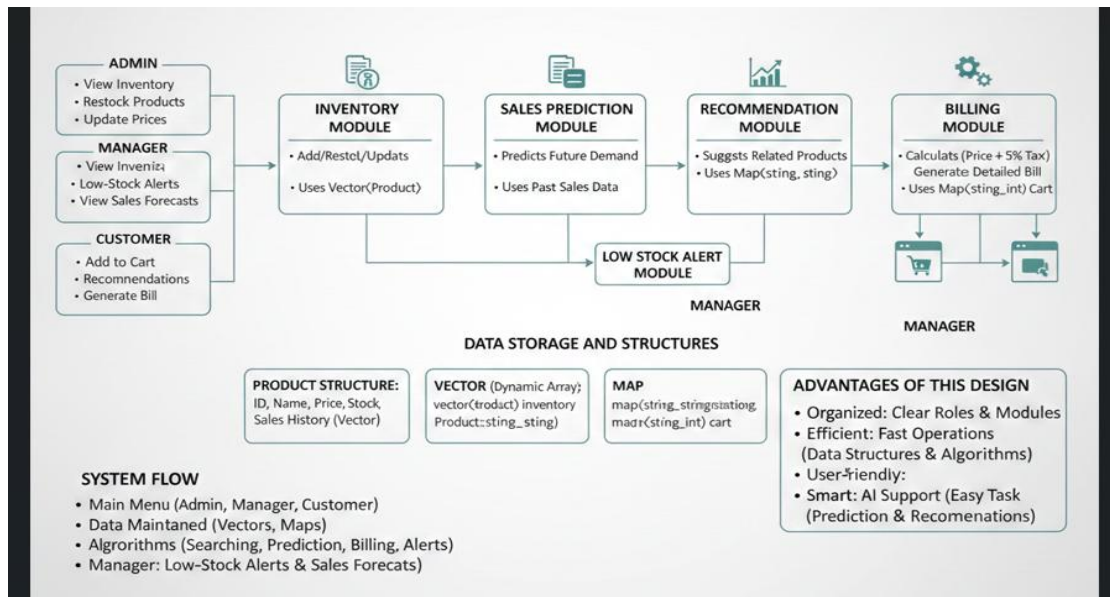


Figure 3 System Flow of AI-based Departmental Store Management System

User Roles:

The system provides three main user roles, each with specific functionalities:

Table 1 Different user roles and their responsibilities in the system.

<u>User roles</u>	<u>Main Responsibilities</u>
Admin	View inventory, restock products, update prices
Manager	Analyze inventory, check low-stock alerts, view sales forecasts
Customer	Browse products, add items to cart, receive recommendations, generate bill

Data Storage and Structures:

The system uses structured data to store and manage information:

✦ Product Structure:

Stores product ID, name, price, stock, and sales history.

✦ Vector:

Vector<Product> inventory stores the list of all products.

✦ Map:

- ◆ map<string, string> recommendations store related product suggestions.
- ◆ map<string, int> cart stores products added by customers with their quantities.

Functional Modules:

The system is divided into functional modules:

✦ Inventory Management Module:

- ◆ Handles addition of new products, restocking, and updating prices.
- ◆ Uses vectors and structures to maintain product details.

✦ Billing Module:

- ◆ Calculates the total price of products in the customer cart.
- ◆ Adds tax (5%) and generates a detailed bill.
- ◆ Ensures fast and accurate billing using maps and iteration algorithms.

✦ Sales Prediction Module:

- ◆ Uses past sales data to predict future demand for each product.
- ◆ Helps managers plan stock accordingly and reduce shortages or overstocking.

✦ Recommendation Module:

- ◆ Suggests related products to customers based on their current selection.
- ◆ Increases sales and improves the shopping experience.

✦ **Low Stock Alert Module:**

- ✦ Checks current stock against predicted demand.
- ✦ Alerts the manager when products are running low.

System Flow:

The system begins with a main menu to select the user role (Admin, Manager, Customer). Each role accesses a specific menu for tasks. Inventory uses vectors and structures, while maps manage recommendations and carts. Algorithms handle searching, sales prediction, billing, and alerts. Customers get product recommendations and automatic billing, while managers receive low-stock alerts and sales forecasts.

Advantages of This Design:

- ✦ **Organized:** Clear separation of roles and modules ensures smooth operation.
- ✦ **Efficient:** Data structures and algorithms make searching, billing, and stock management fast.
- ✦ **User-friendly:** Customers, managers, and admins can easily perform their tasks.
- ✦ **Smart:** Sales prediction and recommendations provide basic AI support to improve business decisions.

IMPLEMENTATION DETAILS:

The **AI-Enhanced Departmental Store Management System** is implemented using the **C++ programming language**. The system is designed in a simple yet efficient manner by using basic data structures and algorithms. The main focus of the implementation is to make the system **organized, easy to use, and reliable** for managing daily store operations such as inventory control, billing, sales analysis, and customer interaction.

The complete system is divided into different modules, where each module performs a specific task. This modular approach improves readability, maintainability, and overall system performance.

Programming Language:

- ✦ C++ is used because it supports **object-oriented features**, strong type checking, and allows the use of **data structures and algorithms** efficiently.

- ✦ It provides **vectors and maps** which are essential for storing dynamic data and key-value pairs.

Data Structures Used:

✦ **Structure (struct):**

- ◆ Used to store all details of a product such as ID, name, price, stock, and sales history.
- ◆ **Example:** struct Product { int id; string name; float price; int stock; vector<int> sales_history; };

✦ **Vector:**

- ◆ vector<Product> inventory stores all products in the store dynamically.
- ◆ Each product's sales_history is also stored as a vector to track previous sales.

✦ **Map:**

- ◆ map<string, string> recommendations store product recommendations for customers.
- ◆ map<string, int> cart stores customer-selected products and their quantities.

Modules Implemented:

The system is divided into three main modules:

✦ **Admin Module:** The Admin module handles store maintenance tasks.

- ◆ View inventory
- ◆ Restock products
- ◆ Update product prices

✦ **Manager Module:** The Manager module focuses on analysis and decision making.

- ◆ View inventory
- ◆ Check low-stock alerts
- ◆ View sales forecasts based on past sales

- ✦ **Customer Module:** The Customer module allows users to interact with the store.
 - ◆ Browse products with prices
 - ◆ Add products to cart
 - ◆ Receive product recommendations
 - ◆ Generate bill including tax automatically

Algorithms Implemented:

- ✦ **Linear Search:** Finds products in the inventory by name.
- ✦ **Sales Prediction:** Predicts future demand based on the last two sales entries.
- ✦ **Recommendation:** Suggests related products using a map.
- ✦ **Billing:** Calculates total cost, adds tax, and prints the bill.
- ✦ **Low Stock Alert:** Checks if current stock is less than predicted need plus buffer and alerts the manager.

Flow of Operations:

- ✦ The system starts and asks the user to select their role (Admin, Manager, Customer).
- ✦ Each user accesses their specific menu to perform tasks.
- ✦ Inventory and sales data are maintained using **vectors and structures**.
- ✦ Customers can browse products, add to cart, and get AI-based recommendations.
- ✦ Managers can analyse sales and check low-stock alerts.
- ✦ Admins can update stock and prices.
- ✦ Billing is automatically generated at checkout for customers.

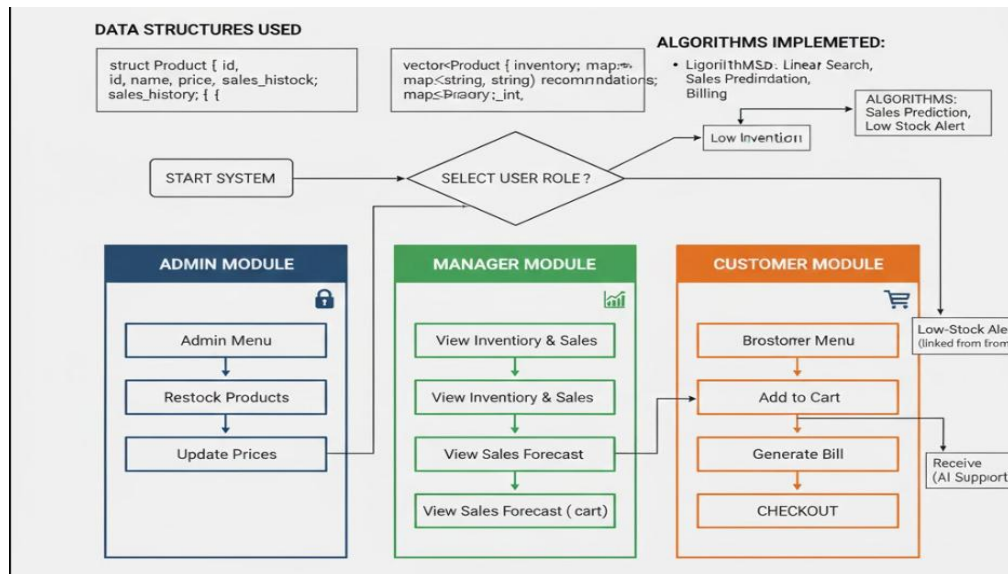
DIAGRAM:

Figure 4 Operational Flow of User Roles

Advantages of Implementation:

- ✦ **Modular:** Each module (Admin, Manager, Customer) works independently.
- ✦ **Efficient:** Uses proper data structures to store and manage data dynamically.
- ✦ **Smart:** Sales predictions and recommendations add basic AI support.
- ✦ **User-Friendly:** Menus and prompts make the system easy to use.
- ✦ **Reliable:** Reduces errors in billing, stock management, and pricing.

TIME AND SPACE COMPLEXITY:

Time and space complexity helps to understand the performance of the system.

Linear Search for Product:

- ✦ **Time Complexity:**
 - ◆ Worst case: $O(n)$, where n is the number of products in the inventory.
 - ◆ Best case: $O(1)$, if the product is the first item.
- ✦ **Space Complexity:** $O(1)$, as no extra memory is needed except variables for index and loop.

Billing:

- ✦ **Time Complexity:** $O(n)$, where n is the number of items in the cart.
- ✦ **Space Complexity:** $O(n)$, due to storing cart items in a map.

Sales Prediction:

- ✦ **Time Complexity:** $O(1)$ for each product, since it uses only the last two entries in the sales history.
- ✦ **Space Complexity:** $O(n)$, where n is the length of sales history vector for each product.

Recommendation:

- ✦ **Time Complexity:** $O(1)$, because it uses a map to find the recommended product quickly.
- ✦ **Space Complexity:** $O(r)$, where r is the number of recommendation pairs stored in the map.

Restocking & Price Update:

- ✦ **Time Complexity:** $O(n)$, because the system searches for the product first using linear search.
- ✦ **Space Complexity:** $O(1)$, as only temporary variables are used.

Viewing Inventory:

- ✦ **Time Complexity:** $O(n)$, where n is the number of products.
- ✦ **Space Complexity:** $O(n)$ for storing the inventory in memory (already in vector).

SAMPLE INPUT & OUTPUT:**Admin Operations:****Input (Admin selects “View Inventory”) & Output:**

```

=== AI STORE MANAGEMENT SYSTEM ===
1-Admin
2-Manager
3-Customer
4-Exit
Choice: 1

--- ADMIN MENU ---
1-View Inventory
2-Restock
3-Update Price
4-Back
Choice: 1

```

ID	Name	Price	Stock
1	Milk	50	25
2	Bread	30	30
3	Eggs	10	50
4	Rice	80	40
5	Oil	200	15
6	Sugar	40	20
7	Tea	120	25
8	Biscuits	25	60
9	Soap	45	35
10	Shampoo	150	10

Input (Admin selects “Restock Product & Updated Price”) & Output:

```

--- ADMIN MENU ---

```

```

1-View Inventory
2-Restock
3-Update Price
4-Back
Choice: 2
Enter product name to restock: Oil
Current stock: 15
Add quantity: 5
Stock updated successfully!

```

```

--- ADMIN MENU ---

```

```

1-View Inventory
2-Restock
3-Update Price
4-Back
Choice: 3
Enter product name: Milk
Current price: 50
New price: 100
Price updated successfully!

```

Manager Operations:**Input (Manager selects “Inventory”) & Output:**

```

=== AI STORE MANAGEMENT SYSTEM ===
1-Admin
2-Manager
3-Customer
4-Exit
Choice: 2

--- MANAGER ANALYTICS ---
1-Inventory
2-Sales Forecast
3-Low Stock Alert
4-Back
Choice: 1

```

ID	Name	Price	Stock
1	Milk	100	25
2	Bread	30	30
3	Eggs	10	50
4	Rice	80	40
5	Oil	200	20
6	Sugar	40	20
7	Tea	120	25
8	Biscuits	25	60
9	Soap	45	35
10	Shampoo	150	10

Input (Manager selects “Sales Forecast & Low Stock Alert”) & Output:

```
--- MANAGER ANALYTICS ---  
1-Inventory  
2-Sales Forecast  
3-Low Stock Alert  
4-Back  
Choice: 2
```

```
--- SALES FORECAST ---  
Milk : 14 units predicted  
Bread : 2 units predicted  
Eggs : 21 units predicted  
Rice : 7 units predicted  
Oil : 5 units predicted  
Sugar : 13 units predicted  
Tea : 6 units predicted  
Biscuits : 16 units predicted  
Soap : 5 units predicted  
Shampoo : 4 units predicted
```

```
--- MANAGER ANALYTICS ---  
1-Inventory  
2-Sales Forecast  
3-Low Stock Alert  
4-Back  
Choice: 3
```

```
--- LOW STOCK ALERT ---  
All stock levels sufficient.
```

Customer Operations:**Input (Customer selects “Browse Products”) & Output:**

```
=== AI STORE MANAGEMENT SYSTEM ===  
1-Admin  
2-Manager  
3-Customer  
4-Exit  
Choice: 3  
  
--- CUSTOMER MENU ---  
1-Browse Products  
2-Add to Cart  
3-Checkout  
Choice: 1  
Milk - Price: 100  
Bread - Price: 30  
Eggs - Price: 10  
Rice - Price: 80  
Oil - Price: 200  
Sugar - Price: 40  
Tea - Price: 120  
Biscuits - Price: 25  
Soap - Price: 45  
Shampoo - Price: 150
```

Input (Customer selects “Add to Cart & Checkout”) & Output:

```
--- CUSTOMER MENU ---
1-Browse Products
2-Add to Cart
3-Checkout
Choice: 2
Product Name: Milk
Quantity: 5
Added! AI Recommendation: Buy Bread
```

```
--- CUSTOMER MENU ---
1-Browse Products
2-Add to Cart
3-Checkout
Choice: 3

----- BILL -----
Milk : 5 x 100 = 500
Tax (5%): 25
Grand Total: 525
-----

Do you want to continue shopping? (Y/N): n

*****
THANKS FOR COMING!
*****
```

TESTING & RESULTS:**Admin Module Testing:**

- ✦ **Test Case 1: View Inventory:**
 - ◆ **Input:** Admin selects “View Inventory”
 - ◆ **Result:** All products displayed accurately
- ✦ **Test Case 2: Restock Product:**
 - ◆ **Input:** Restock 10 units of Milk
 - ◆ **Result:** Stock updated correctly
- ✦ **Test Case 3: Update Price:**
 - ◆ **Input:** Change price of Bread from 30 → 35
 - ◆ **Result:** Price updated successfully

Manager Module Testing:

- ✦ **Test Case 1: Sales Forecast:**
 - ◆ **Input:** Manager selects “Sales Forecast”
 - ◆ **Result:** Forecast values match expected calculation

✦ **Test Case 2: Low Stock Alert:**

- ✦ **Input:** Check products with stock $\leq$ predicted demand + 5
- ✦ **Result:** Alerts displayed correctly

Customer Module Testing:

✦ **Test Case 1: Browse Products:**

- ✦ **Input:** Customer selects “Browse Products”
- ✦ **Result:** Products displayed correctly

✦ **Test Case 2: Add to Cart & Recommendation:**

- ✦ **Input:** Add 2 units of Milk to cart
- ✦ **Result:** Cart updated and recommendation correct

✦ **Test Case 3: Checkout & Billing:**

- ✦ **Input:** Generate bill for cart items
- ✦ **Result:** Bill generated correctly with tax included

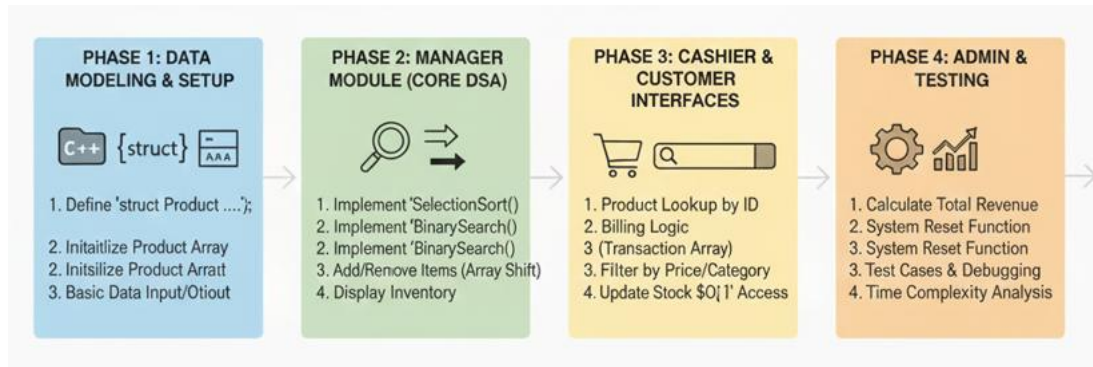
CONCLUSION:

The **AI-based Departmental Store Management System** is designed to manage a departmental store efficiently using **C++**, **data structures**, and **simple algorithms**. The system provides three distinct roles: Admin, Manager, and Customer, each with their own set of functionalities.

- ✦ **Admin:** Can view inventory, restock products, and update prices easily.
- ✦ **Manager:** Can analyse sales, receive low-stock alerts, and forecast future sales to make informed decisions.
- ✦ **Customer:** Can browse products, add items to the cart, get AI-based product recommendations, and generate accurate bills including tax.

The system uses **structures**, **vectors**, and **maps** to efficiently manage product data, inventory, sales history, recommendations, and customer carts. Key algorithms such as linear search, sales prediction, billing, recommendations, and low-stock alerts make the system fast,

reliable, and easy to use. Testing showed accurate billing, correct stock updates, reliable sales forecasts, and effective recommendations.



FUTURE WORK:

- ✦ Add database support for permanent data storage and multi-user access.
- ✦ Improve AI features for better recommendations and customer behaviour prediction.
- ✦ Replace console menus with a user-friendly GUI (Windows or Web).
- ✦ Develop a mobile app for browsing products and placing orders.
- ✦ Add reports and analytics for sales, stock, and decision making.

REFERENCES:

Books:

- ✦ **Data Structures and Algorithms in C++ from Adam Drozdek:** Covers arrays, vectors, maps, structures, and basic algorithms used in your project.
- ✦ **Data Structures Using C++ from D. S. Malik:** Explains practical implementation of linked lists, stacks, queues, and arrays in C++.
- ✦ **C++ Programming: From Problem Analysis to Program Design from D. S. Malik:** Good reference for designing console-based applications using structured programming.
- ✦ **Introduction to Algorithms from Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein:** For understanding algorithms like search, prediction, and basic computational analysis.

Websites:

- ✦ [Programiz – C++ Programming](#): Easy explanations of C++ concepts, data structures, and console-based programs.
- ✦ [GeeksforGeeks – C++ STL](#): Focused on Standard Template Library, including vectors, maps, and basic algorithms.
- ✦ [TutorialsPoint – Data Structures](#): Covers all major data structures like arrays, stacks, queues, and their applications.
- ✦ [LearnCpp.com](#): Step-by-step C++ tutorials, including structures, arrays, and object-oriented programming concepts.

Appendix A: Source Code of AI-based Departmental Store Management System:

```
#include <iostream>

#include <string>

#include <vector>

#include <map>

#include <algorithm>

using namespace std;

struct Product {

    int id; string name; float price; int stock; vector<int> sales_history; };

vector<Product> inventory; map<string, string> recommendations;

int find_product(string name) {

    for (int i = 0; i < (int)inventory.size(); i++) {

        if (inventory[i].name == name)

            return i; } return -1; }

string recommend(string product) {

    if (recommendations.count(product))

        return recommendations[product]; return "No recommendation"; }

int predict_sales(Product &p) {

    int n = p.sales_history.size(); if (n < 2) return 0;

    int slope = p.sales_history[n - 1] - p.sales_history[n - 2];

    int pred = p.sales_history[n - 1] + slope * 2; return max(0, pred); }

void view_inventory() {

    cout << "\nID\tName\t\tPrice\tStock\n";

    cout << "-----\n";

    for (int i = 0; i < (int)inventory.size(); i++) {

        cout << inventory[i].id << "\t"
```

```
        << inventory[i].name << (inventory[i].name.length() < 8 ? "\t\t" :
"\t")    << inventory[i].price << "\t" << inventory[i].stock << "\n"; } }

void restock_product() {

    string name; int amount; cout << "Enter product name to restock: ";

    cin >> ws; getline(cin, name); int idx = find_product(name);

    if (idx == -1) {

        cout << "Product not found!\n"; return; }

    cout << "Current stock: " << inventory[idx].stock << "\nAdd quantity: ";

    cin >> amount; inventory[idx].stock += amount;

    cout << "Stock updated successfully!\n"; }

void update_price() {

    string name; float price; cout << "Enter product name: "; cin >> ws;
    getline(cin, name); int idx = find_product(name); if (idx == -1) {

        cout << "Product not found!\n"; return; }

    cout << "Current price: " << inventory[idx].price << "\nNew price: ";

    cin >> price; inventory[idx].price = price; cout << "Price updated
successfully!\n"; }

void generate_bill(map<string, int> &cart) {

    if (cart.empty()) { cout << "Cart empty.\n"; return; }

    float total = 0; cout << "\n----- BILL ----- \n";

    for (map<string, int>::iterator it = cart.begin(); it != cart.end(); ++it) {

        int idx = find_product(it->first); if (idx != -1) {

            float cost = it->second * inventory[idx].price; total += cost;

            cout << it->first << " : " << it->second << " x "

                << inventory[idx].price << " = " << cost << "\n"; } }

    float tax = total * 0.05; cout << "Tax (5%): " << tax << "\n";

    cout << "Grand Total: " << total + tax << "\n"; cout << "----- \n";

    cart.clear(); } void admin_menu() { while (true) {
```

```
    cout << "\n--- ADMIN MENU ---";

    cout << "\n1-View Inventory\n2-Restock\n3-Update Price\n4-Back\nChoice: ";

    int c; cin >> c; if (c == 1) view_inventory(); else if (c == 2)
restock_product(); else if (c == 3) update_price(); else if (c == 4) break; }}

void manager_menu() { while (true) { cout << "\n--- MANAGER ANALYTICS ---";

    cout << "\n1-Inventory\n2-Sales Forecast\n3-Low Stock Alert\n4-
Back\nChoice: "; int c; cin >> c; if (c == 1) view_inventory(); else if (c == 2) {

        cout << "\n--- SALES FORECAST ---\n";

        for (int i = 0; i < (int)inventory.size(); i++) {

            cout << inventory[i].name << " : " << predict_sales(inventory[i])
<< " units predicted\n"; }}else if (c == 3){cout << "\n--- LOW STOCK ALERT ---\n";

            bool found = false; for (int i = 0; i < (int)inventory.size(); i++) {

                int p = predict_sales(inventory[i]);

                if (inventory[i].stock <= p + 5) {

                    cout << "! " << inventory[i].name << " (Stock: " <<
inventory[i].stock << " | Predicted Need: " << p << ")\n";found = true; }}

                if (!found) cout << "All stock levels sufficient.\n"; }

            else if (c == 4) break; } }

void customer_menu() {

    map<string, int> cart; char choice_continue; while (true) {

        cout << "\n--- CUSTOMER MENU ---";

        cout << "\n1-Browse Products\n2-Add to Cart\n3-Checkout\nChoice: ";

        int c; cin >> c; if (c == 1) {

            for (int i = 0; i < (int)inventory.size(); i++) {

                cout << inventory[i].name << " - Price: " << inventory[i].price <<
"\n";    }    }

            else if (c == 2) { string p; int q;

                cout << "Product Name: "; cin >> ws; getline(cin, p);

                cout << "Quantity: "; cin >> q; int idx = find_product(p);
```

```
        if (idx == -1 || q > inventory[idx].stock) {

            cout << "Not available!\n"; } else {

                inventory[idx].stock -= q; cart[p] += q;

                cout << "Added! AI Recommendation: Buy " << recommend(p)<< "\n";}}

    else if (c == 3) { generate_bill(cart);

        cout << "\nDo you want to continue shopping? (Y/N): ";

        cin >> choice_continue; if (choice_continue == 'n' || choice_continue
== 'N') {            cout << "\n*****";

                cout << "\n  THANKS FOR COMING!  ";

                cout << "\n*****\n"; break; }  }

    else cout << "Invalid choice!\n";  } } void init_data() { Product p;

    p.id = 1; p.name = "Milk"; p.price = 50; p.stock = 25;

    p.sales_history.clear(); p.sales_history.push_back(5);
p.sales_history.push_back(8); inventory.push_back(p);

    p.id = 2; p.name = "Bread"; p.price = 30; p.stock = 30;

    p.sales_history.clear(); p.sales_history.push_back(8);
p.sales_history.push_back(6); inventory.push_back(p);

    p.id = 3; p.name = "Eggs"; p.price = 10; p.stock = 50;

    p.sales_history.clear(); p.sales_history.push_back(12);
p.sales_history.push_back(15); inventory.push_back(p);

    p.id = 4; p.name = "Rice"; p.price = 80; p.stock = 40;

    p.sales_history.clear(); p.sales_history.push_back(4);
p.sales_history.push_back(5);  inventory.push_back(p);

    p.id = 5; p.name = "Oil"; p.price = 200; p.stock = 15;

    p.sales_history.clear(); p.sales_history.push_back(2);
p.sales_history.push_back(3); inventory.push_back(p);

    p.id = 6; p.name = "Sugar"; p.price = 40; p.stock = 20;

    p.sales_history.clear(); p.sales_history.push_back(7);
p.sales_history.push_back(9); inventory.push_back(p);
```

```
p.id = 7; p.name = "Tea"; p.price = 120; p.stock = 25;

p.sales_history.clear(); p.sales_history.push_back(3);
p.sales_history.push_back(4); inventory.push_back(p);

p.id = 8; p.name = "Biscuits"; p.price = 25; p.stock = 60;

p.sales_history.clear(); p.sales_history.push_back(10);
p.sales_history.push_back(12); inventory.push_back(p);

p.id = 9; p.name = "Soap"; p.price = 45; p.stock = 35;

p.sales_history.clear(); p.sales_history.push_back(5);
p.sales_history.push_back(5); inventory.push_back(p);

p.id = 10; p.name = "Shampoo"; p.price = 150; p.stock = 10;

p.sales_history.clear(); p.sales_history.push_back(1);
p.sales_history.push_back(2); inventory.push_back(p);

recommendations["Milk"] = "Bread"; recommendations["Bread"] = "Eggs";
recommendations["Eggs"] = "Milk"; recommendations["Rice"] = "Oil";
recommendations["Oil"] = "Rice"; recommendations["Sugar"] = "Tea";
recommendations["Tea"] = "Biscuits"; recommendations["Biscuits"] = "Tea";
recommendations["Soap"] = "Shampoo"; recommendations["Shampoo"] = "Soap";}

int main() {

    init_data(); while (true) { cout << "\n=== AI STORE MANAGEMENT SYSTEM ===\n";

        cout << "1-Admin\n2-Manager\n3-Customer\n4-Exit\nChoice: ";

        int c; cin >> c; if (c == 1) admin_menu();

        else if (c == 2) manager_menu();

        else if (c == 3) customer_menu();

        else if (c == 4) break; else cout << "Invalid choice!\n";

    }

    return 0;

}
```

THE END